Московский государственный технический университет
имени Н.Э. Баумана

Кафедра «Системы обработки информации и управления»

Ю.Е. Гапанюк

СЕМИНАР №3
ОСНОВЫ ОБРАБОТКИ ФАЙЛОВ
В ФОРМАТЕ «CSV»

Учебные материалы по дисциплине
«Архитектура автоматизированных систем обработки информации и управления»
Профиль: «Архитектура программных систем»

Москва – 2026

Оглавление

1 ЦЕЛЬ 3

2 ФОРМАТ CSV 3

3 РЕЛЯЦИОННАЯ АЛГЕБРА, ЯЗЫК SQL 3

4 РАБОТА С SQLITE В C# 4

5 РЕАЛИЗАЦИЯ ПРОТОТИПА РЕЛЯЦИОННОЙ СУБД НА ЧИСТОМ C# 12

1 Цель

Написать консольное приложение, которое реализует базовые операции реляционной алгебры на примере файлов CSV

2 Формат CSV

Формат CSV (формат с разделителями-запятыми) представляет собой способ построчного хранения табличных данных, в каждой строке поля разделены с помощью разделителя, по умолчанию это запятая.

Существует детальное описание формата – https://ru.wikipedia.org/wiki/CSV

В первой заголовочной строке файла как правило хранятся имена полей (колонок). Тип колонок явно в этом формате не хранится и должен автоматически определяться по типу данных колонки.

3 Реляционная алгебра, язык SQL

Алгеброй называется множество (носитель) с возможными операциями, которые замкнуты на этом носителе. То есть результатом применения операции будет элемент, принадлежащий носителю. Более детально этот подход рассмотрен в статье – https://habr.com/ru/articles/655059/ .

Это условие выполняется для реляционной алгебры. В реляционной алгебре носителем являются «отношения» или неформально «таблицы» – https://ru.wikipedia.org/wiki/Реляционная_модель_данных. Файлы CSV содержат такие таблицы.

В реляционной алгебре выделяют следующие виды операций:

- Переименование
- Объединение
- Пересечение
- Вычитание

- Операция присваивания
- Декартово произведение
- Выборка (ограничение)
- Проекция
- Соединение
- Деление

В результате выполнения каждой операции получается отношение (таблица), к которой можно применять другие операции.

На практике вместо работы с чистыми операциями используется язык SQL – https://ru.wikipedia.org/wiki/SQL.

Система Управления Базами Данных (или СУБД – https://ru.wikipedia.org/wiki/Система_управления_базами_данных) которая хранит данные в виде таблиц и поддерживает язык SQL для их обработки называется реляционной СУБД.

В рамках данного занятия мы поработаем с реляционной СУБД SQLite (https://ru.wikipedia.org/wiki/SQLite) а затем попробуем написать очень простой прототип реляционной СУБД, который поддерживает несколько базовых операций реляционной алгебры.

Необходимо отметить, что в настоящее время в проектах по анализу данных широко используется СУБД DuckDB (https://en.wikipedia.org/wiki/DuckDB) но для нее существует мало библиотек для .NET, в основном DuckDB используется в Python.

4 Работа с SQLite в C#

Для установки поддержки SQLite в проекте необходимо выполнить следующие действия:

1. Откройте меню «Сервис» (или «Средства») → «Диспетчер пакетов NuGet» → «Консоль диспетчера пакетов» (Tools → NuGet Package Manager → Package Manager Console)

2. В открывшейся внизу консоли введите команду: Install-Package Microsoft.Data.Sqlite

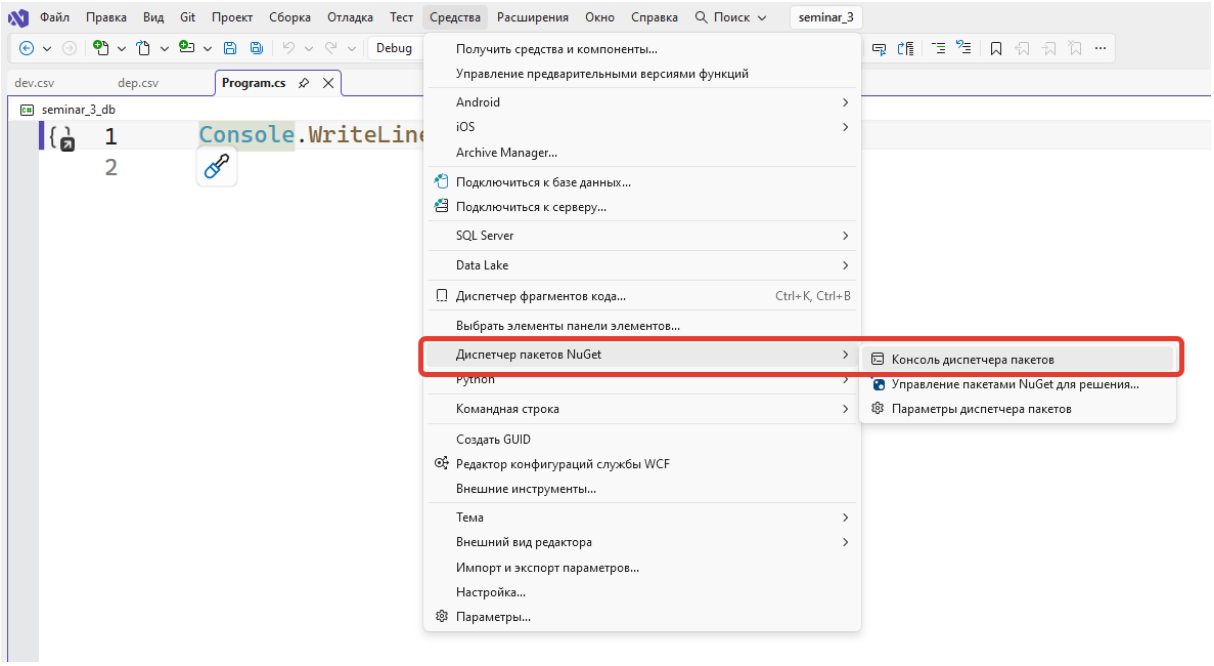

Рис. 1. Открытие консоли диспетчера пакетов (шаг 1).

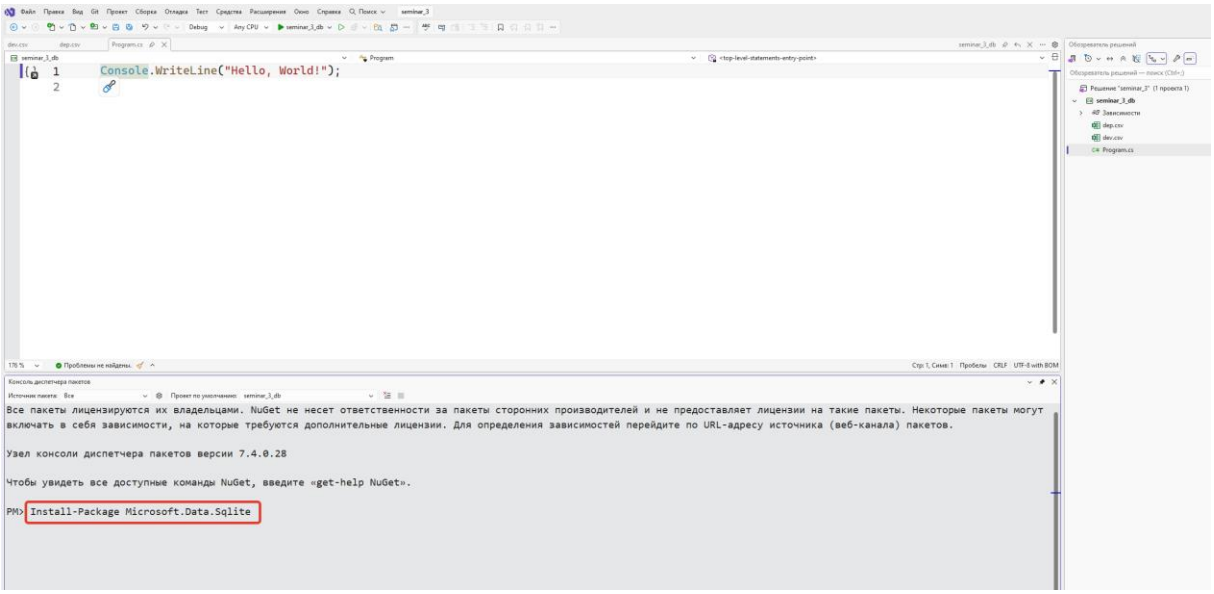

Рис. 2. Ввод команды (шаг 2).

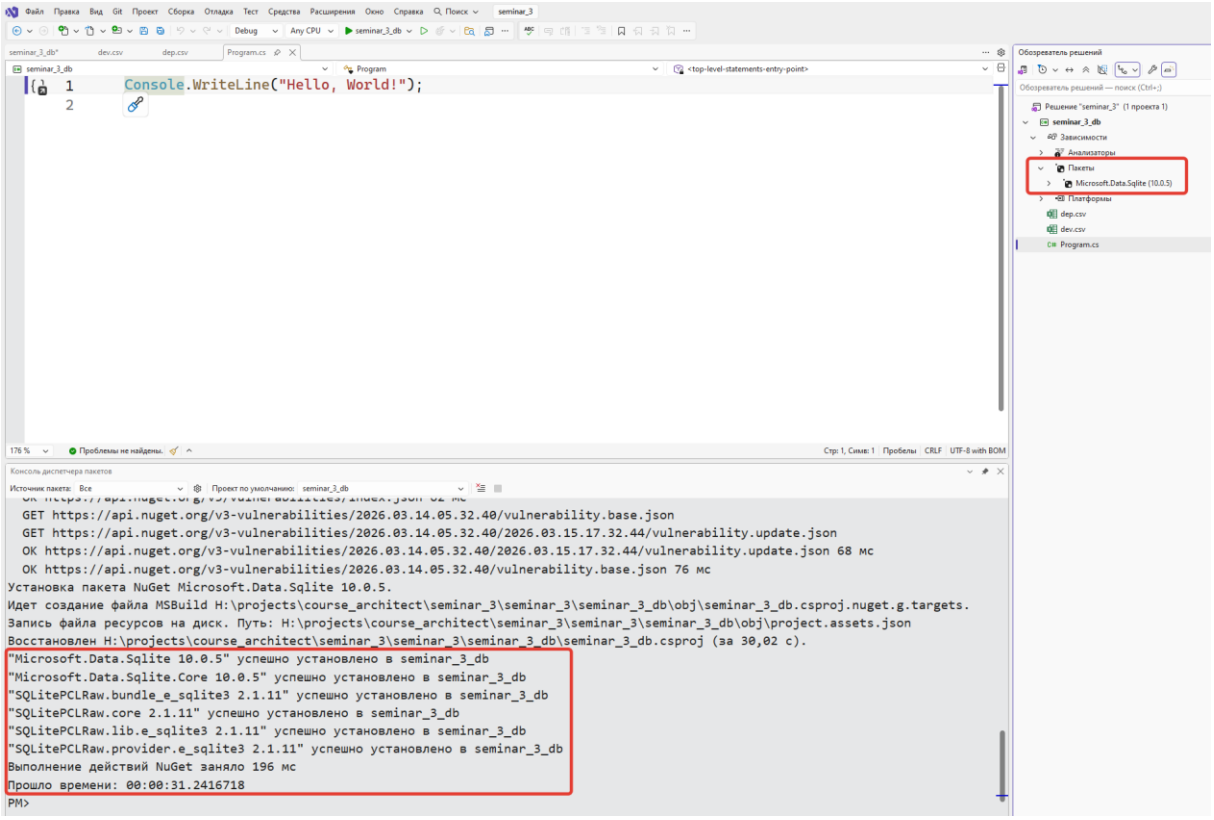

Рис. 3. Результат выполнения команды (шаг 2).

Обратите внимание, что установка пакетов NuGet всегда происходит не в решение (solution), а в текущий проект решения.

Текст программы:

```
using Microsoft.Data.Sqlite;

// === Точка входа (операторы верхнего уровня) ===

const string dbFile = "developers.db";
const string devCsv = "dev.csv";
const string depCsv = "dep.csv";

CreateDatabase(dbFile);
LoadData(dbFile, devCsv, depCsv);

PrintData(dbFile, "dep");
PrintData(dbFile, "dev");

// Проекция — получаем список значений одной колонки
List<string> names = Projection(dbFile, "dev", "dev_name");
Console.WriteLine("\n=== Результат Projection(dev, dev_name) ===");
foreach (var name in names)
    Console.WriteLine(name);

// Выборка — получаем строки, подходящие под условие
List<string[]> rows = Where(dbFile, "dev", "dep_id", "2");
Console.WriteLine("\n=== Результат Where(dev, dep_id, 2) ===");
foreach (var row in rows)
    Console.WriteLine(string.Join(" | ", row));
```

```
// Соединение
var (columns, joinRows) = Join(dbFile, "dev", "dep", "dep_id", "dep_id");
Console.WriteLine("\n=== Результат Join(dev, dep, dep_id, dep_id) ===");
Console.WriteLine(string.Join(" | ", columns));
Console.WriteLine(new string('-', 80));
foreach (var row in joinRows)
    Console.WriteLine(string.Join(" | ", row));

// Среднее количество коммитов по отделам
var (gavgCols, gavgRows) = GroupAvg(dbFile, "dev", "dep_id", "dev_commits");
Console.WriteLine("\n=== Результат GroupAvg(dev, dep_id, dev_commits) ===");
Console.WriteLine(string.Join(" | ", gavgCols));
Console.WriteLine(new string('-', 40));
foreach (var row in gavgRows)
    Console.WriteLine(string.Join(" | ", row));


// ================================================
// === Статические функции ========================
// ================================================

static void CreateDatabase(string dbPath)
{
    if (File.Exists(dbPath))
        File.Delete(dbPath);

    using var connection = new SqliteConnection($"Data Source={dbPath}");
    connection.Open();

    var command = connection.CreateCommand();

    command.CommandText = @"
        CREATE TABLE dep (
            dep_id   INTEGER PRIMARY KEY,
            dep_name TEXT NOT NULL
        );";
    command.ExecuteNonQuery();

    command.CommandText = @"
        CREATE TABLE dev (
            dev_id      INTEGER PRIMARY KEY,
            dep_id      INTEGER NOT NULL,
            dev_name    TEXT    NOT NULL,
            dev_commits INTEGER NOT NULL,
            FOREIGN KEY (dep_id) REFERENCES dep(dep_id)
        );";
    command.ExecuteNonQuery();

    Console.WriteLine($"[OK] База данных «{dbPath}» создана, таблицы dep и dev
готовы.");
}

static void LoadData(string dbPath, string devCsvPath, string depCsvPath)
{
    using var connection = new SqliteConnection($"Data Source={dbPath}");
    connection.Open();

    using (var transaction = connection.BeginTransaction())
    {
        var lines = File.ReadAllLines(depCsvPath);
        for (int i = 1; i < lines.Length; i++)
        {
            var parts = lines[i].Split(';');
            if (parts.Length < 2) continue;

            var cmd = connection.CreateCommand();
```

```
            cmd.CommandText = "INSERT INTO dep (dep_id, dep_name) VALUES (@id,
@name);";
            cmd.Parameters.AddWithValue("@id", int.Parse(parts[0]));
            cmd.Parameters.AddWithValue("@name", parts[1]);
            cmd.ExecuteNonQuery();
        }
        transaction.Commit();
        Console.WriteLine($"[OK] Загружено строк из «{depCsvPath}»: {lines.Length -
1}");
    }

    using (var transaction = connection.BeginTransaction())
    {
        var lines = File.ReadAllLines(devCsvPath);
        for (int i = 1; i < lines.Length; i++)
        {
            var parts = lines[i].Split(';');
            if (parts.Length < 4) continue;

            var cmd = connection.CreateCommand();
            cmd.CommandText = @"INSERT INTO dev (dev_id, dep_id, dev_name,
dev_commits)
                                VALUES (@devId, @depId, @name, @commits);";
            cmd.Parameters.AddWithValue("@devId", int.Parse(parts[0]));
            cmd.Parameters.AddWithValue("@depId", int.Parse(parts[1]));
            cmd.Parameters.AddWithValue("@name", parts[2]);
            cmd.Parameters.AddWithValue("@commits", int.Parse(parts[3]));
            cmd.ExecuteNonQuery();
        }
        transaction.Commit();
        Console.WriteLine($"[OK] Загружено строк из «{devCsvPath}»: {lines.Length -
1}");
    }
}

static void PrintData(string dbPath, string tableName)
{
    using var connection = new SqliteConnection($"Data Source={dbPath}");
    connection.Open();

    var cmd = connection.CreateCommand();
    cmd.CommandText = $"SELECT * FROM {tableName} ORDER BY 1;";

    using var reader = cmd.ExecuteReader();

    int columnCount = reader.FieldCount;
    const int colWidth = 20;

    Console.WriteLine($"\n========== Таблица {tableName} ==========");

    for (int c = 0; c < columnCount; c++)
        Console.Write($"{reader.GetName(c),-colWidth}");
    Console.WriteLine();
    Console.WriteLine(new string('-', colWidth * columnCount));

    while (reader.Read())
    {
        for (int c = 0; c < columnCount; c++)
            Console.Write($"{reader.GetValue(c),-colWidth}");
        Console.WriteLine();
    }
}

/// <summary>
/// Проекция: возвращает список значений одной колонки.
/// SELECT columnName FROM tableName
/// </summary>
```

```
static List<string> Projection(string dbPath, string tableName, string columnName)
{
    var result = new List<string>();

    using var connection = new SqliteConnection($"Data Source={dbPath}");
    connection.Open();

    var cmd = connection.CreateCommand();
    cmd.CommandText = $"SELECT {columnName} FROM {tableName} ORDER BY 1;";

    using var reader = cmd.ExecuteReader();
    while (reader.Read())
    {
        result.Add(reader.GetValue(0).ToString()!);
    }

    return result;
}

/// <summary>
/// Выборка: возвращает строки, где значение колонки равно заданному.
/// SELECT * FROM tableName WHERE columnName = value
/// </summary>
static List<string[]> Where(string dbPath, string tableName, string columnName,
string value)
{
    var result = new List<string[]>();

    using var connection = new SqliteConnection($"Data Source={dbPath}");
    connection.Open();

    var cmd = connection.CreateCommand();
    cmd.CommandText = $"SELECT * FROM {tableName} WHERE {columnName} = @val ORDER BY
1;";
    cmd.Parameters.AddWithValue("@val", value);

    using var reader = cmd.ExecuteReader();
    while (reader.Read())
    {
        var row = new string[reader.FieldCount];
        for (int c = 0; c < reader.FieldCount; c++)
            row[c] = reader.GetValue(c).ToString()!;
        result.Add(row);
    }

    return result;
}

static (string[] columns, List<string[]> rows) Join(
    string dbPath,
    string table1, string table2,
    string key1, string key2)
{
    var rows = new List<string[]>();

    using var connection = new SqliteConnection($"Data Source={dbPath}");
    connection.Open();

    var cmd = connection.CreateCommand();
    cmd.CommandText = $@"
        SELECT *
        FROM {table1}
        INNER JOIN {table2}
            ON {table1}.{key1} = {table2}.{key2}
        ORDER BY 1;";

    using var reader = cmd.ExecuteReader();
```

```
    // Имена столбцов
    var columns = new string[reader.FieldCount];
    for (int c = 0; c < reader.FieldCount; c++)
        columns[c] = reader.GetName(c);

    // Строки данных
    while (reader.Read())
    {
        var row = new string[reader.FieldCount];
        for (int c = 0; c < reader.FieldCount; c++)
            row[c] = reader.GetValue(c).ToString()!;
        rows.Add(row);
    }

    return (columns, rows);
}


static (string[] columns, List<string[]> rows) GroupAvg(
    string dbPath,
    string tableName,
    string groupColumn,
    string avgColumn)
{
    var rows = new List<string[]>();

    using var connection = new SqliteConnection($"Data Source={dbPath}");
    connection.Open();

    var cmd = connection.CreateCommand();
    cmd.CommandText = $@"
        SELECT {groupColumn}, AVG({avgColumn}) AS avg_{avgColumn}
        FROM {tableName}
        GROUP BY {groupColumn}
        ORDER BY 1;";

    using var reader = cmd.ExecuteReader();

    var columns = new string[reader.FieldCount];
    for (int c = 0; c < reader.FieldCount; c++)
        columns[c] = reader.GetName(c);

    while (reader.Read())
    {
        var row = new string[reader.FieldCount];
        for (int c = 0; c < reader.FieldCount; c++)
            row[c] = reader.GetValue(c).ToString()!;
        rows.Add(row);
    }

    return (columns, rows);
}
```

```
Консоль отладки Microsoft Visual Studio
[OK] База данных <developers.db> создана, таблицы dep и dev готовы.
[OK] Загружено строк из <dep.csv>: 3
[OK] Загружено строк из <dev.csv>: 9

========== Таблица dep ==========
dep_id              dep_name
----------------------------------------
1                   Отдел C++
2                   Отдел C#
3                   Отдел Python

========== Таблица dev ==========
dev_id              dep_id              dev_name            dev_commits
--------------------------------------------------------------------------------
1                   1                   Иванов И.И.         3
2                   1                   Петров П.П.         5
3                   1                   Сидоров С.С.        8
4                   2                   Коломнин К.К.       30
5                   2                   Суздальцев С.С.     50
6                   2                   Рязанцев Р.Р.       80
7                   3                   Волгин В.В.         300
8                   3                   Енисеев Е.Е.        500
9                   3                   Иртышев И.И.        800

=== Результат Projection(dev, dev_name) ===
Волгин В.В.
Енисеев Е.Е.
Иванов И.И.
Иртышев И.И.
Коломнин К.К.
Петров П.П.
Рязанцев Р.Р.
Сидоров С.С.
Суздальцев С.С.

=== Результат Where(dev, dep_id, 2) ===
4 | 2 | Коломнин К.К. | 30
5 | 2 | Суздальцев С.С. | 50
6 | 2 | Рязанцев Р.Р. | 80

=== Результат Join(dev, dep, dep_id, dep_id) ===
dev_id | dep_id | dev_name | dev_commits | dep_id | dep_name
--------------------------------------------------------------------------------
1 | 1 | Иванов И.И. | 3 | 1 | Отдел C++
2 | 1 | Петров П.П. | 5 | 1 | Отдел C++
3 | 1 | Сидоров С.С. | 8 | 1 | Отдел C++
4 | 2 | Коломнин К.К. | 30 | 2 | Отдел C#
5 | 2 | Суздальцев С.С. | 50 | 2 | Отдел C#
6 | 2 | Рязанцев Р.Р. | 80 | 2 | Отдел C#
7 | 3 | Волгин В.В. | 300 | 3 | Отдел Python
8 | 3 | Енисеев Е.Е. | 500 | 3 | Отдел Python
9 | 3 | Иртышев И.И. | 800 | 3 | Отдел Python

=== Результат GroupAvg(dev, dep_id, dev_commits) ===
dep_id | avg_dev_commits
----------------------------------------
1 | 5,333333333333333
2 | 53,333333333333336
3 | 533,3333333333334
```

Рис. 4. Результат выполнения программы.

5 Реализация прототипа реляционной СУБД на чистом C#

Для того чтобы в окне консоли корректно отображалась кодировка Unicode (UTF-8), необходимо включить ее с помощью команды

```
chcp 65001
```

Текст программы:

```
// === Точка входа (операторы верхнего уровня) ===

string mode = args[0].ToLower();

switch (mode)
{
    case "projection":
        {
            // CSV поступает из стандартного ввода
            var table = ReadCsv(Console.In, ';');
            string columnName = args[1];
            var result = Projection(table, columnName);
            WriteCsv(Console.Out, result, ';');
            break;
        }

    case "where":
        {
            if (args.Length < 3)
            {
                Console.Error.WriteLine(
                    "Ошибка: режим 'where' требует два параметра: where <колонка>
<значение>");
                return 1;
            }
            // CSV поступает из стандартного ввода
            var table = ReadCsv(Console.In, ';');
            string columnName = args[1];
            string value = args[2];
            var result = Where(table, columnName, value);
            WriteCsv(Console.Out, result, ';');
            break;
        }

    case "join":
        {
            if (args.Length < 5)
            {
                Console.Error.WriteLine(
                    "Использование: program join <таблица1> <таблица2> <ключ1>
<ключ2>");
                return 1;
            }

            using var reader1 = File.OpenText(args[1] + ".csv");
            using var reader2 = File.OpenText(args[2] + ".csv");

            var left = ReadCsv(reader1, ';');
            var right = ReadCsv(reader2, ';');

            var result = Join(left, right, args[3], args[4]);
            WriteCsv(Console.Out, result, ';');
            break;
```

```
        }

    case "group_avg":
        {
            if (args.Length < 3)
            {
                Console.Error.WriteLine(
                    "Использование: program group_avg <колонка_группировки>
<колонка_значений>");
                return 1;
            }
            // CSV поступает из стандартного ввода
            var table = ReadCsv(Console.In, ';');

            string groupColumn = args[1];
            string valueColumn = args[2];

            var result = GroupAvg(table, groupColumn, valueColumn);
            WriteCsv(Console.Out, result, ';');
            break;
        }

}

return 0;

// ================================================
// === Статические функции ========================
// ================================================

// ---------- Чтение / запись CSV ----------

static CsvTable ReadCsv(TextReader reader, char separator)
{
    string? headerLine = reader.ReadLine();
    if (headerLine is null)
        throw new InvalidOperationException("Входной поток пуст — нет строки
заголовков.");

    string[] headers = headerLine.Split(separator);
    var rows = new List<CsvRow>();

    string? line;
    while ((line = reader.ReadLine()) is not null)
    {
        if (string.IsNullOrWhiteSpace(line))
            continue;

        string[] parts = line.Split(separator);
        rows.Add(new CsvRow(parts));
    }

    return new CsvTable(headers, rows);
}

static void WriteCsv(TextWriter writer, CsvTable table, char separator)
{
    writer.WriteLine(string.Join(separator, table.Headers));

    foreach (var row in table.Rows)
        writer.WriteLine(string.Join(separator, row.Fields));
}

// ---------- Вспомогательные ----------

/// <summary>
/// Ищет индекс столбца по имени. Бросает исключение, если столбец не найден.
```

```
/// </summary>
static int FindColumnIndex(CsvTable table, string columnName)
{
    int index = Array.IndexOf(table.Headers, columnName);
    if (index < 0)
        throw new ArgumentException(
            $"Колонка «{columnName}» не найдена. " +
            $"Доступные колонки: {string.Join(", ", table.Headers)}");
    return index;
}

// ---------- Операции ----------

/// <summary>
/// Проекция: SELECT columnName FROM table.
/// Возвращает таблицу с одним столбцом.
/// </summary>
static CsvTable Projection(CsvTable table, string columnName)
{
    int colIndex = FindColumnIndex(table, columnName);

    string[] newHeaders = [columnName];
    var newRows = new List<CsvRow>();

    foreach (var row in table.Rows)
    {
        string[] fields = [row.Fields[colIndex]];
        newRows.Add(new CsvRow(fields));
    }

    return new CsvTable(newHeaders, newRows);
}

/// <summary>
/// Выборка: SELECT * FROM table WHERE columnName = value.
/// Возвращает таблицу с теми же столбцами, но отфильтрованными строками.
/// </summary>
static CsvTable Where(CsvTable table, string columnName, string value)
{
    int colIndex = FindColumnIndex(table, columnName);

    var newRows = new List<CsvRow>();

    foreach (var row in table.Rows)
    {
        if (row.Fields[colIndex] == value)
        {
            newRows.Add(row);
        }
    }

    return new CsvTable(table.Headers, newRows);
}


/// <summary>
/// Inner Join: для каждой пары строк, где left[leftKey] == right[rightKey],
/// склеивает поля обеих строк в одну.
/// </summary>
static CsvTable Join(CsvTable left, CsvTable right,
                     string leftKey, string rightKey)
{
    int leftKeyIndex = FindColumnIndex(left, leftKey);
    int rightKeyIndex = FindColumnIndex(right, rightKey);

    // Заголовки: все колонки левой + все колонки правой
    var newHeaders = new string[left.Headers.Length + right.Headers.Length];
```

```
    for (int i = 0; i < left.Headers.Length; i++)
        newHeaders[i] = left.Headers[i];
    for (int i = 0; i < right.Headers.Length; i++)
        newHeaders[left.Headers.Length + i] = right.Headers[i];

    var newRows = new List<CsvRow>();

    foreach (var leftRow in left.Rows)
    {
        foreach (var rightRow in right.Rows)
        {
            if (leftRow.Fields[leftKeyIndex] == rightRow.Fields[rightKeyIndex])
            {
                // Склеиваем поля двух строк
                var fields = new string[leftRow.Fields.Length +
rightRow.Fields.Length];
                for (int i = 0; i < leftRow.Fields.Length; i++)
                    fields[i] = leftRow.Fields[i];
                for (int i = 0; i < rightRow.Fields.Length; i++)
                    fields[leftRow.Fields.Length + i] = rightRow.Fields[i];

                newRows.Add(new CsvRow(fields));
            }
        }
    }

    return new CsvTable(newHeaders, newRows);
}

/// <summary>
/// Вычисляет среднее арифметическое массива чисел.
/// </summary>
static double Average(List<double> values)
{
    double sum = 0;
    for (int i = 0; i < values.Count; i++)
        sum += values[i];
    return sum / values.Count;
}

/// <summary>
/// Группировка с вычислением среднего:
/// SELECT groupColumn, AVG(valueColumn) FROM table GROUP BY groupColumn
/// </summary>
static CsvTable GroupAvg(CsvTable table, string groupColumn, string valueColumn)
{
    int groupIndex = FindColumnIndex(table, groupColumn);
    int valueIndex = FindColumnIndex(table, valueColumn);

    // Ключ — значение колонки группировки,
    // значение — список числовых значений в этой группе
    var groups = new Dictionary<string, List<double>>();

    foreach (var row in table.Rows)
    {
        string key = row.Fields[groupIndex];
        double value = double.Parse(row.Fields[valueIndex]);

        if (!groups.ContainsKey(key))
            groups[key] = new List<double>();

        groups[key].Add(value);
    }

    // Формируем результат
    string[] newHeaders = [groupColumn, "avg_" + valueColumn];
    var newRows = new List<CsvRow>();
```

```
    foreach (var pair in groups)
    {
        string avg = Average(pair.Value).ToString("F2");
        newRows.Add(new CsvRow([pair.Key, avg]));
    }

    return new CsvTable(newHeaders, newRows);
}


// ================================================
// === Записи (records) для представления данных ===
// ================================================

/// <summary>
/// Одна строка CSV-файла. Хранит значения полей в порядке столбцов.
/// record даёт неизменяемость и готовый ToString() для отладки.
/// </summary>
record CsvRow(string[] Fields);

/// <summary>
/// CSV-таблица целиком: заголовки (имена столбцов) + список строк.
/// </summary>
record CsvTable(string[] Headers, List<CsvRow> Rows);
```

```
Командная строка

H:\projects\course_architect\seminar_3\seminar_3\tdb\bin\Debug\net10.0>tdb where dep_id 2 < dev.csv | tdb projection dev_name
dev_name
Коломнин К.К.
Суздальцев С.С.
Рязанцев Р.Р.

H:\projects\course_architect\seminar_3\seminar_3\tdb\bin\Debug\net10.0>tdb join dev dep dep_id dep_id
dev_id;dep_id;dev_name;dev_commits;dep_id;dep_name
1;1;Иванов И.И.;3;1;Отдел C++
2;1;Петров П.П.;5;1;Отдел C++
3;1;Сидоров С.С.;8;1;Отдел C++
4;2;Коломнин К.К.;30;2;Отдел C#
5;2;Суздальцев С.С.;50;2;Отдел C#
6;2;Рязанцев Р.Р.;80;2;Отдел C#
7;3;Волгин В.В.;300;3;Отдел Python
8;3;Енисеев Е.Е.;500;3;Отдел Python
9;3;Иртышев И.И.;800;3;Отдел Python

H:\projects\course_architect\seminar_3\seminar_3\tdb\bin\Debug\net10.0>tdb join dev dep dep_id dep_id < dev.csv | tdb projection dev_name
dev_name
Иванов И.И.
Петров П.П.
Сидоров С.С.
Коломнин К.К.
Суздальцев С.С.
Рязанцев Р.Р.
Волгин В.В.
Енисеев Е.Е.
Иртышев И.И.

H:\projects\course_architect\seminar_3\seminar_3\tdb\bin\Debug\net10.0>tdb group_avg dep_id dev_commits < dev.csv
dep_id;avg_dev_commits
1;5,33
2;53,33
3;533,33

H:\projects\course_architect\seminar_3\seminar_3\tdb\bin\Debug\net10.0>tdb group_avg dep_id dev_commits < dev.csv > tmp.csv

H:\projects\course_architect\seminar_3\seminar_3\tdb\bin\Debug\net10.0>tdb join tmp dep dep_id dep_id
dep_id;avg_dev_commits;dep_id;dep_name
1;5,33;1;Отдел C++
2;53,33;2;Отдел C#
3;533,33;3;Отдел Python

H:\projects\course_architect\seminar_3\seminar_3\tdb\bin\Debug\net10.0>
```

Рис. 5. Результат выполнения программы.